```
# Printing a Sudoku grid
def print_sudoku(grid):
    for row in grid:
        print(" ".join(map(str, row)))

# Checking if a move is valid in the Sudoku grid
def is_valid_move(grid, row, col, num):
    if num in grid[row]:
        return False  # If the number exists in the same row, it's not a valid
         move
    elif num in [grid[i][col] for i in range(9)]:
        return False  # If the number exists in the same column, it's not a
         valid move
    start_row, start_col = 3 * (row // 3), 3 * (col // 3)  # Find the start of
     the 3x3 subgrid
    for i in range(start_row, start_row + 3):
        for j in range(start_col, start_col + 3):
            if grid[i][j] == num:
                return False  # If the number exists in the 3x3 subgrid, it's
                 not a valid move
    return True  # The move is valid

# Solving a Sudoku puzzle using the backtracking algorithm
def solve_sudoku(grid):
    for row in range(9):
        for col in range(9):
            if grid[row][col] == 0:  # Check if the cell is empty (0
             represents empty cell)
                for num in range(1, 10):  # Try numbers from 1 to 9
                    if is_valid_move(grid, row, col, num):
                        grid[row][col] = num  # Try placing the number
                        if solve_sudoku(grid):  # Recursively solve the puzzle
                            return True
                        grid[row][col] = 0  # If no solution is found, reset
                         the cell
                return False  # If no valid number can be placed, backtrack to
                 a previous state
    return True  # If all cells are filled, the puzzle is solved

# Reading a Sudoku puzzle from a text file
def read_sudoku_file(filename):
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()  # Read all lines from the file
            sudoku_grid = []  # Initialize an empty list to store the Sudoku
             grid

            # Iterate through each line in the file
            for line in lines:
                row = [int(num) for num in line.strip().split()]  # Convert
                 line to a list of integers
```

```
                sudoku_grid.append(row)  # Append the row to the Sudoku grid
                 list
        return sudoku_grid  # Return the Sudoku grid as a list of lists
    except FileNotFoundError:
        print(f"File '{filename}' not found.")  # Handle the case where the
         file is not found
        return None  # Return None to indicate an error

# Getting the filename for the Sudoku puzzle from the user
filename = input("Enter the name of the Sudoku puzzle file: ")  # Prompt the
 user for a filename prompt the user for a filename
sudoku_grid = read_sudoku_file(filename)  # Call the read_sudoku_file function
 with the users filename

if sudoku_grid is not None:
    print("Sudoku puzzle from file:")
    print_sudoku(sudoku_grid)

    # Solving the Sudoku puzzle
    if solve_sudoku(sudoku_grid):
        print("\nSolved Sudoku:")
        print_sudoku(sudoku_grid)
    else:
        print("\nNo solution found.")
```